Reporte

Equipo 16	
Nombre: Lesly Adamaris Cotlame Nieto Christopher Bethancourt Contreras Francisco Mora Ruiz	**Matrícula**: 2226958 2226948 2226912
Nombre del curso: Componentes de lógica de negocios.	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Módulo Netacad: PIA, apartado 3: Implementación	**Actividad**: Manual de entrega continua y despliegue (CD) de Eventos Nova - procedimiento preparado; despliegue pendiente.
Fecha: 05/11/2026 (revisión documental) · Entrega oficial del PIA: 06/11/2026	

Bibliografía (Formato APA)

- Docker Inc. (s. f.). Docker Compose [Documentación oficial]. https://docs.docker.com/compose/
- Humble, J., & Farley, D. (2010). Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley.
- Render. (s. f.). Blueprint YAML reference. https://render.com/docs/blueprint-spec
- Wiggins, A. (2017). The twelve-factor app. https://12factor.net

Objetivo:

Documentar la implementación de Eventos Nova con instrucciones reales y verificables para instalar, configurar, ejecutar, probar, integrar y desplegar el sistema, sin incluir secretos ni afirmar operaciones externas que no tienen evidencia.

Procedimiento:

Manual de entrega continua y despliegue (CD) - Eventos Nova

1. Alcance y estado

Este manual deja preparado el procedimiento para desplegar Eventos Nova en Render (u otra plataforma compatible con Docker) con Swagger UI activo para la demostración. Al 05/11/2026 Render no está desplegado, no existe URL pública y render.yaml es un borrador sin aplicar. El despliegue que este manual preveía para el 04/11/2026 no se realizó.

Elemento	Situación
render.yaml	Borrador sin aplicar: base PostgreSQL 18 (postgresMajorVersion "18"), servicio web Docker con preDeployCommand «python -m alembic upgrade head», healthCheckPath /ready, worker; autoDeployTrigger checksPass; plan starter
Despliegue en Render	No realizado
URL pública	No existe; no se registra ninguna
Swagger UI en producción	Pendiente; requiere EVENTOSNOVA_DOCS=1 definido a mano en el panel
Costo	Los planes de la base y los servicios pueden tener costo; requiere aprobación del equipo

2. Plataforma

La plataforma documentada en el repositorio es **Render** (blueprint `render.yaml`): un servicio web Docker `eventosnova-api`, un servicio worker `eventosnova-worker` y una base PostgreSQL gestionada `eventosnova-db`. El PIA admite también Railway, Fly.io o equivalentes; el procedimiento es el mismo en esencia (imagen Docker, migraciones previas, API y trabajador separados, variables de entorno). Los nombres y costos de los planes deben verificarse en la plataforma antes de aplicarlos; el blueprint advierte que el plan gratuito de la base caduca y que el del servicio web se suspende por inactividad.

3. Requisitos previos

N.º	Requisito
1	Repositorio GitHub con el código, los workflows y los archivos Docker versionados (ver Manual de CI y documento del repositorio).

N.º	Requisito
2	Ejecución de CI en verde sobre el commit a desplegar (render.yaml usa autoDeployTrigger: checksPass, que solo despliega cuando pasan las comprobaciones del commit).
3	Cuenta en la plataforma con autorización del equipo y método de pago si se usan planes de pago.
4	Correo y nombre para el primer administrador; contraseña robusta (mínimo 12 caracteres) guardada fuera del repositorio.

4. Configuración y variables de entorno

Variable	Servicio	Valor
DATABASE_URL	web y worker	fromDatabase eventosnova-db (connectionString); el código normaliza postgres:// a postgresql+psycopg://
EVENTOSNOVA_PERSISTENCIA	web y worker	postgresql
EVENTOSNOVA_ENTORNO	web y worker	produccion
FORWARDED_ALLOW_IPS	web	* (confianza en el proxy TLS de la plataforma)
EVENTOSNOVA_DOCS	web	1 (necesaria para Swagger UI en la demostración; agregar, no está en el borrador)
PORT	web	La inyecta la plataforma; el Dockerfile escucha en ${PORT:-8000}

No se define `EVENTOSNOVA_DISPONIBILIDAD_ARCHIVO` (prohibida) ni ninguna contraseña en el repositorio. Con `EVENTOSNOVA_ENTORNO=produccion` la cookie es Secure, se envía HSTS y no puede desactivarse la autenticación.

5. Construcción

La plataforma construye la imagen con el Dockerfile sin indicar etapa: desde el commit 7aee2cb la etapa final y predeterminada es runtime (hereda de app: python:3.13-slim, usuario no root eventosnova, CMD uvicorn con ${PORT:-8000}). Alternativamente, el flujo publicar-imagen.yml construye la misma imagen y la publica en GitHub Container Registry al crear una etiqueta v*.

```
# verificación local opcional antes de publicar
docker build --target app -t eventosnova-app:verificacion .
docker compose -f docker-compose.instalacion-limpia.yml up -d --build   #
requiere LIMPIA_PG_CLAVE temporal
```

6. Publicación

```
git tag v1.0.0
git push origin v1.0.0        # dispara publicar-imagen.yml (GHCR) y el deploy
hook si existe
```

En Render: New → Blueprint, seleccionar el repositorio y aplicar `render.yaml` revisando los planes; o crear manualmente los tres recursos con los mismos parámetros.

7. Migraciones

El servicio web ejecuta `python -m alembic upgrade head` como `preDeployCommand`, antes de arrancar cada versión. La base debe quedar en la revisión 0009; el trabajador comprueba las migraciones al iniciar.

8. Inicio de servicios

N.º	Servicio
1	Web: CMD del Dockerfile (uvicorn api.main:app --app-dir src --host 0.0.0.0 --port ${PORT:-8000}) con healthCheckPath /ready.
2	Worker: python -m infrastructure.persistencia.trabajador_outbox --intervalo 2 (necesario para el inicio automático del evento y el despacho del Outbox).
3	Primer administrador, desde la consola (shell) del servicio web:

```
export ADMIN_CLAVE_TEMPORAL='<definir en la consola, no en el repositorio>'
python scripts/crear_administrador.py --base eventosnova --correo <correo> \
    --nombre "<nombre>" --servidor-remoto --contrasena-desde-entorno
ADMIN_CLAVE_TEMPORAL
unset ADMIN_CLAVE_TEMPORAL
```

9. Comprobaciones posteriores

#	Comprobación	Resultado esperado
1	GET <URL>/health	200
2	GET <URL>/ready	200 (503 indica base no disponible)
3	GET <URL>/docs	Swagger UI con 77 operaciones (requiere EVENTOSNOVA_DOCS=1)
4	GET <URL>/openapi.json	OpenAPI 3.1.0
5	POST <URL>/auth/inicio-sesion con el administrador	200 y cookie Secure
6	GET <URL>/app/	Pantalla de ingreso
7	Crear un Coordinador y registrar un evento de prueba	201; el evento queda En gestión
8	Registros del worker	Ciclos sin errores cada 2 s

10. Swagger y verificación de disponibilidad

Para la demostración se usa Swagger UI en <URL>/docs. Se recomienda abrir la URL 10 minutos antes de la presentación, porque los planes que se suspenden por

inactividad tardan en reanudarse. Las operaciones protegidas requieren iniciar sesión primero; las escrituras necesitan el token CSRF devuelto.

11. Procedimiento de reversión

N.º	Paso
1	Si falla la construcción o el preDeploy: la versión anterior sigue activa; corregir y volver a desplegar.
2	Si la nueva versión falla después de publicarse: en Render, Deploys → seleccionar el despliegue anterior → «Rollback»; o desplegar de nuevo la etiqueta anterior.
3	Las migraciones son aditivas: una versión anterior de la aplicación puede ejecutarse con el esquema 0009. No ejecutar alembic downgrade sobre datos reales sin respaldo.
4	Antes de cambios riesgosos, generar un respaldo de la base gestionada desde la plataforma.
5	Si la plataforma no está disponible el día de la demostración: ejecutar docker compose up -d --build localmente y mostrar Swagger en http://127.0.0.1:8000/docs, declarando que es el entorno local.

12. Registro de evidencias del despliegue

Evidencia	Estado al 05/11/2026
Commits del Bloque A publicados y CI en verde	Pendiente
URL pública del sistema	Pendiente (no se registra ninguna)
Captura de /docs en producción	Pendiente
Captura de /ready y /health	Pendiente
Registro del preDeploy (alembic upgrade head)	Pendiente
Registro del worker	Pendiente
Primer administrador creado en producción	Pendiente
Commit desplegado	Pendiente
Actualización del README, reporte final y presentación con la URL	Pendiente